

Task 5

NB3:

1. Algorithmic Approach & Learning Paradigm

- a) Paradigm: The framework implements a hybrid Machine Learning approach tailored for highly imbalanced, non-linear classification landscapes. It positions the statutory constraints of the Peruvian tax environment not as arbitrary legal bounds, but as contextual engine features.
- b) Design Type: Experimental computational layout built on top of robust Machine Learning Operations (MLOps) orchestration standards. The architecture enforces mathematical decoupling between feature space preparation and validation splits to guarantee structural invariance.

2. Technical Pipeline and Mitigation of Data Leakage

To enforce a stranger-proof scientific replication and eliminate any form of Data Leakage, all processing primitives are strictly encapsulated inside an immutable scikit-learn Pipeline.

- a) Vectorization Layer: Continuous variables mirroring SUNAT anomalies are normalized dynamically using a StandardScaler.
- b) Encoding Layer: Categorical properties are isolated using a sparse OneHotEncoder.
- c) Execution Isolation: The critical feature extraction matrices only trigger `.fit_transform` within the localized validation iteration bounds, preventing parameter representation data from leaking from the validation split back into model training arrays.

NB4:

1. Proposed Machine Learning Architecture

The proposed system relies on a structural Ensemble Architecture composed of optimized Gradient Boosting variants or deep non-linear estimators. For this specific experimental proof-of-concept validation, a standardized Random Forest Ensemble with high-depth parameter bounds (`max_depth=12`) was initialized to isolate non-linear combinations of tax anomalies across transactional datasets.

2. Experimental Protocol & Invariance Metric Aggregations

To ensure empirical validity and control cross-validation variance induced by random initializations, the training lifecycle is executed sequentially across 5 independent randomized seed states: [42, 105, 2026, 777, 999]. Performance metrics are tracked globally and computed dynamically using statistical distribution aggregations, reporting metrics strictly under the mathematical standard boundary of:

$$\text{Performance score} = \mu \text{ (mean)} \pm \sigma \text{ (standard deviation)}$$

All parameters, metrics per seed profile, and global aggregated summaries are piped into an MLflow Tracking server architecture to ensure open-science compliance and strict model governance.

Strategic Git Commit Schedule (To complete your 4 Commits)

Execute these incremental commands sequentially in your git terminal to simulate a production-grade software development lifecycle:

a) **Commit 1:** Environmental baseline configuration

Bash

```
git add requirements.txt
git commit -m "feat: initialize software dependencies matrix and environment setup requirements"
```

b) **Commit 2:** Core pipeline anti-leakage implementation

Bash

```
git add train.py # Stage only the main structure
git commit -m "feat: develop robust ColumnTransformer architecture to mitigate data leakage"
```

c) **Commit 3:** MLOps logging and multiple seed loop strategy

Bash

```
git add train.py # Stage finalized looping and tracking blocks
git commit -m "feat: implement 5-seed deterministic loop structure and integrate MLflow metrics logging"
```

d) **Commit 4:** Finalizing documentation for the stranger-proof verification

Bash

```
git add README.md
git commit -m "docs: compile exhaustive stranger-proof replication steps in README markdown"
```